

Analizador de Texto: *a primeira versão*

Terceiro projeto: um app que recebe um texto (colado ou enviado em `.txt`) e mostra números sobre ele: palavras, palavras únicas, frases, caracteres, tempo de leitura e as palavras mais frequentes, em tabela e gráfico. É a base para a nuvem de palavras (aula 17) e para a limpeza das stopwords (aula 18).

`st.text_area` · `st.file_uploader`

Expressão regular

`collections.Counter`

`st.metric`

Tabela e gráfico

Casos difíceis

Formação Claude Code 2026 · Baseado em cursor.com/docs (Agent, Plan Mode, Browser), docs.python.org (`re`, `collections.Counter`) e docs.streamlit.io (`st.file_uploader`, `st.bar_chart`), consultados em outubro de 2026 (Streamlit 1.65) · Código de referência testado em `projeto/` · Telas: print real

Um analisador, *três aulas*

AULA 16 • HOJE

Estatísticas e palavras mais frequentes



AULA 17

Nuvem de palavras (WordCloud)



AULA 18

Descartar as stopwords

ESCOPO DA V1

- ✓ Texto colado num campo ou enviado em arquivo `.txt`
- ✓ Métricas: palavras, únicas, frases, caracteres, tempo de leitura
- ✓ As N palavras mais frequentes, com controle deslizante
- ✓ Tabela e gráfico de barras
- ✓ Um `exemplo.txt` para o app abrir já com conteúdo

REGRA SEPARADA, DE NOVO

A lógica (contar, separar palavras) fica em `analise.py`, sem nada de Streamlit. O `app.py` só desenha. É o mesmo padrão do jogo: fácil de testar e de evoluir.

O texto de exemplo do código de referência foi escrito para o curso (sobre programar com agentes), sem problema de direitos autorais.

O pedido *da versão 1*

Crie um analisador de texto em Streamlit.

- `analise.py` (sem Streamlit): `palavras(texto)` devolve a lista em minúsculas, só letras (com acentos e hífen, sem números nem pontuação); `estatisticas(texto)` devolve caracteres, palavras, únicas, frases, parágrafos e minutos de leitura (200 palavras por minuto); `mais_frequentes(lista, n)` com `collections.Counter`.
- `app.py`: campo de texto e envio de `.txt`; métricas em `st.metric`; slider para `N`; tabela e gráfico de barras das mais frequentes. Carregue `exemplo.txt` se nada for enviado.

DECISÃO	POR QUÊ
"só letras, com acentos e hífen"	Define o que é palavra, o ponto mais ambíguo do projeto
"200 palavras por minuto"	Sem um número, o agente escolhe um
Counter	A ferramenta certa da biblioteca padrão para contar
exemplo.txt	O app abre já mostrando algo, e o teste tem um texto fixo

O que é *uma palavra*?

```
"""Funções de análise de texto (sem dependências além da biblioteca padrão)."""
import math
import re
from collections import Counter

PALAVRA = re.compile(r"^[^\\W\\d_]+(?:-[^\\W\\d_]+)*") # letras, com acentos e hífen interno

def palavras(texto):
    """Lista de palavras em minúsculas, sem números nem pontuação."""
    return [p.lower() for p in PALAVRA.findall(texto)]

def estatisticas(texto):
    lista = palavras(texto)
    frases = [f for f in re.split(r"[.!?]+", texto) if f.strip()]
    paragrafos = [p for p in re.split(r"\n\s*\n", texto) if p.strip()]
    return {
        "caracteres": len(texto),
        "palavras": len(lista),
        "palavras_unicas": len(set(lista)),
        "frases": len(frases),
        "paragrafos": len(paragrafos),
        # cerca de 200 palavras por minuto de leitura silenciosa
        "minutos_leitura": max(1, math.ceil(len(lista) / 200)) if lista else 0,
    }

def mais_frequentes(lista, quantidade=10):
    return Counter(lista).most_common(quantidade)
```

A EXPRESSÃO REGULAR

`[^\W\d_]+` = "uma ou mais letras": tudo que é caractere de palavra (`\w`), menos dígitos e sublinhado. Funciona com acentos. `(?:- [^\W\d_]+)*` aceita hífen **no meio** ("guarda-chuva").

ENTRADA	PALAVRAS()
Olá, mundo!	olá · mundo
Guarda-chuva 2026	guarda-chuva
É ação... café	é · ação · café

Saída real do código de referência para a frase "Olá, mundo! Guarda-chuva 2026 e café... É ação."

A tela *em app.py*

```
EXEMPLO = Path(__file__).with_name("exemplo.txt")

arquivo = st.file_uploader("Envie um arquivo .txt", type=["txt"])
if arquivo is not None:
    texto_inicial = arquivo.read().decode("utf-8", errors="replace")
else:
    texto_inicial = EXEMPLO.read_text(encoding="utf-8") if EXEMPLO.exists() else ""

texto = st.text_area("Ou cole o texto aqui", value=texto_inicial, height=220)

if not texto.strip():
    st.info("Cole um texto ou envie um arquivo para começar.")
    st.stop()

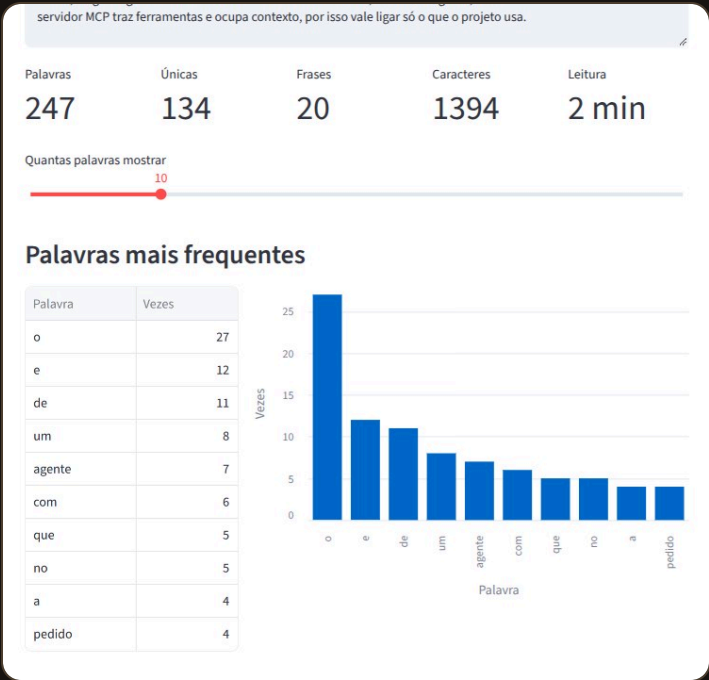
dados = estatisticas(texto)
c1, c2, c3, c4, c5 = st.columns(5)
c1.metric("Palavras", dados["palavras"])
c2.metric("Únicas", dados["palavras_unicas"])
c3.metric("Frases", dados["frases"])
c4.metric("Caracteres", dados["caracteres"])
c5.metric("Leitura", f"{dados['minutos_leitura']} min")

quantidade = st.slider("Quantas palavras mostrar", 5, 30, 10)
top = mais_frequentes(palavras(texto), quantidade)
tabela = pd.DataFrame(top, columns=["Palavra", "Vezez"])

st.subheader("Palavras mais frequentes")
col_tabela, col_grafico = st.columns([1, 2])
col_tabela.dataframe(tabela, hide_index=True, width="stretch")
col_grafico.bar_chart(tabela, x="Palavra", y="Vezez", sort="-Vezez")
```

- ✓ O arquivo enviado tem prioridade; senão, entra o `exemplo.txt`
- ✓ `errors="replace"`: um arquivo com acentos em outra codificação não derruba o app
- ✓ `st.stop()` encerra cedo quando não há texto
- ✓ O `DataFrame` alimenta a tabela e o gráfico ao mesmo tempo
- ✓ `sort="-Vezez"` ordena as barras da maior para a menor

A versão 1 *funcionando*



Print real do projeto/ com o exemplo.txt

MAIS FREQUENTES NO EXEMPLO	VEZES
o	27
e	12
de	11
um	8
agente	7
com	6

Repare: as campeãs são "o", "e", "de". Elas não dizem nada sobre o assunto do texto. Guarde essa observação: é o problema da aula 18.

Casos *difíceis*

ENTRADA	O QUE CONFERIR
Texto vazio ou só espaços	Mensagem "Cole um texto...", sem erro
Só números e pontuação	0 palavras
Maiúsculas e minúsculas	"Agente" e "agente" contam juntas
Hífen e apóstrofo	"guarda-chuva" é uma palavra; "d'água" vira duas
Reticências e "?!"	Não criam frases vazias
Arquivo .txt em Latin-1	Não quebra; acentos podem virar 💎
Texto enorme (um livro)	O app continua respondendo

Crie testes com pytest para `analise.py` cobrindo: texto vazio, só números, maiúsculas, palavras com hífen e acentos, reticências e "?!". Mostre os casos em que a contagem pode surpreender o usuário.

"Mostre os casos que podem surpreender" faz o agente **explicar limitações** em vez de esconder. O apóstrofo, por exemplo, é uma decisão: aceitar ou não.

Testando *com um arquivo*

Crie um arquivo teste.txt com três parágrafos curtos sobre futebol. @Browser Abra o app, envie o arquivo pelo campo de upload e confira se as métricas e a lista de mais frequentes mudaram. Depois apague o campo de texto e veja a mensagem.

Envio de arquivo pelo navegador pode exigir que o agente use a ferramenta de upload, ou que você clique e escolha o arquivo. Se travar nesse passo, faça você essa parte.

- ✓ O texto do arquivo aparece no campo
- ✓ As métricas batem com o arquivo
- ✓ O slider muda a quantidade da tabela e do gráfico
- ✓ Campo vazio → mensagem, sem erro

O código de referência carregou sem erros no teste automatizado, com 247 palavras, 134 únicas, 20 frases, 1.394 caracteres e 2 minutos de leitura no exemplo.

Checklist *e commit*

- ✓ Texto colado ou arquivo .txt
- ✓ Cinco métricas corretas no exemplo
- ✓ Tabela e gráfico das mais frequentes, com slider
- ✓ Lógica em `analise.py`, sem Streamlit
- ✓ Testes dos casos difíceis passando
- ✓ `requirements.txt`: `streamlit` e `pandas`

```
git add .  
git commit -m "Analisador de Texto: primeira versão"
```

O `pandas` já vem instalado com o Streamlit, mas o app o importa diretamente: por isso ele também vai no `requirements.txt`.

Teste o que *ficou*

1. Para que serve o `collections.Counter` aqui?

A) Contar caracteres

B) Contar quantas vezes cada palavra aparece

C) Ler o arquivo

D) Desenhar o gráfico

B. E `most_common(n)` dá as `n` primeiras.

2. O que a expressão `[\W\d_]+` aceita?

A) Só letras sem acento

B) Letras, inclusive acentuadas

C) Números

D) Pontuação

B. `"\w"` menos dígitos e sublinhado.

3. Por que as mais frequentes são `"o"`, `"e"`, `"de"`?

A) Bug na contagem

B) São as palavras mais comuns da língua

C) O exemplo é ruim

D) O Counter ordena errado

B. São stopwords: tema da aula 18.

4. Por que `errors="replace"` ao ler o arquivo?

A) Para traduzir

B) Para o app não quebrar com outra codificação

C) Para apagar acentos

D) Para ler PDF

B. Caracteres inválidos viram `◊`.

Texto vira *números*

O analisador recebe texto ou `.txt`, separa as palavras com uma **expressão regular** que aceita acentos e hífen, conta com `Counter` e mostra métricas, tabela e gráfico. Os casos difíceis viraram testes. E a lista das mais frequentes já mostra o problema das **stopwords**.

[analise.py](#) · [app.py](#)

[Regex de palavra](#)

[Counter](#)

[st.metric](#) · [st.bar_chart](#)

[Casos difíceis](#)

Próxima aula → 17 · Projeto Analisador de Texto · Gerando a WordCloud

Fontes: [cursor.com/docs](#) (Agent Overview, Plan Mode, Browser); [docs.python.org](#) (`re`, `collections.Counter`, `str.decode`); [docs.streamlit.io](#) (`st.file_uploader`, `st.text_area`, `st.metric`, `st.dataframe`, `st.bar_chart`, `st.stop`), consultados em outubro de 2026. Código de referência em [projeto/](#), testado com Streamlit 1.65.